```
print "{$me} and {$friend} are going for a pint." . PHP_EOL;
```

```
Output:
paul and Łukasz are going for a pint.
```

This time we're using the built-in function `strtolower()`, which will transform uppercase characters in a string to lowercase. Except that here it only worked on my name: Łukasz should have been transformed to łukasz, but we can see from the output that this has not happened. The reason for this is that `strtolower()` explicitly states (in the official documentation) that it will only work for a limited subset of ASCII characters (A–Z).

Clearly, neither of the two string functions above is capable of handling multi-byte text, but fortunately for us, PHP offers multi-byte versions of the vast majority via the `mbstring` module. Most PHP installations will have this module installed by default. You can verify its presence on your system with the following command:

```
$ php -m
[PHP Modules]
...
mbstring
```

This module offers versions of the core string functions that are aware of multiple Unicode-based encodings, not just UTF-8, and also utility functions for detecting and converting the encoding of string data. In the next section of this chapter, we'll be looking at the core string manipulation functions (and their multi-byte counterparts), and we'll cover the multi-byte utility functions in a following section. Let's take a quick look at the multi-byte safe versions of our last two examples; we'll swap `strlen()` and `strtolower()` for `mb_strlen()` and `mb_strtolower()`:

```
$lat = 'ABCDE';
$pol = 'AĄBCĆ';

var_dump(mb_strlen($lat));
var_dump(mb_strlen($pol));
```

```
Output:
int(5)
int(5)
```